

Collegium Witelona
Wydział nauk technicznych i ekonomicznych
Projektowanie i programowanie systemów internetowych I



Sprawozdanie z pracy projektowej

System biblioteczno-sklepowy *meow*
ASP.NET Core MVC + REST API

Zespół projektowy

Filip Jędryczkowski, Kyrylo Shchedrin, Nikola Pisarska

nr albumu: 44917

grupa: s2PAM2(2)u

Legnica, 2026

Spis treści

1	Opis przedmiotu zlecenia	2
1.1	Zakres funkcjonalny	2
2	Opis technologiczny proponowanego rozwiązania	2
2.1	Architektura ogólna	2
2.2	Stos technologiczny	3
2.3	Model danych	3
2.4	Warstwa API i cache	3
2.5	Uwierzytelnianie i autoryzacja	4
2.6	Lokalizacja i interfejs	4
2.7	Integracje zewnętrzne (stan implementacji)	4
3	Wykaz zadań projektowych z szacowanym czasem	4
4	Podział zadań w zespole	5
5	Instrukcja uruchomienia systemu	5
5.1	Wymagania wstępne	5
5.2	Uruchomienie lokalne	6
5.3	Uruchomienie zdalne (produkcja)	6
5.4	Pipeline CI	7
6	Dokumentacja OpenAPI	7
6.1	Dostęp do dokumentacji	7
6.2	Endpointy	8
6.3	Przykład wywołania	8
7	Podsumowanie	8

1 Opis przedmiotu zlecenia

Przedmiotem zlecenia projektowego jest zaprojektowanie, implementacja i wdrożenie zintegrowanego systemu informatycznego łączącego **wypożyczalnię książek** oraz **sklep internetowy** pod wspólną marką *meow*.

Zgodnie z wymaganiami prowadzącego przedmiot *Projektowanie i programowanie systemów internetowych I* system musi udostępniać:

- **interfejs webowy** dla klientów (przeglądanie katalogu, koszyk, checkout, rezerwacja egzemplarzy bibliotecznych, konto użytkownika),
- **panel administracyjny** do zarządzania katalogiem, klientami, wypożyczeniami i zamówieniami sklepowymi,
- **REST API** (z dokumentacją OpenAPI) jako warstwę danych dla planowanej aplikacji mobilnej.

1.1 Zakres funkcjonalny

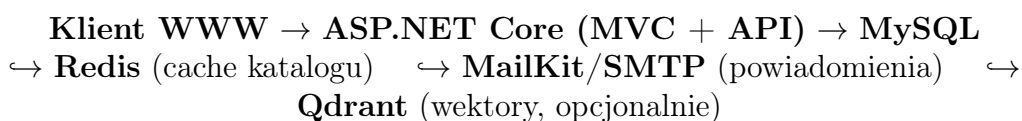
System obejmuje m.in.:

- rejestrację i logowanie użytkowników (role: klient, administrator),
- zarządzanie książkami, egzemplarzami bibliotecznymi i stanem magazynowym sklepu,
- proces zakupu (koszyk, dane dostawy, składanie zamówienia),
- wypożyczenia (termin 30 dni), rezerwacje, zwroty oraz obsługę opóźnień,
- eksport danych raportowych z modułu wypożyczeń (`ReportsController`, widok finansowy),
- integrację z zewnętrznymi usługami (cache Redis, Brevo API / Mailpit, szkielet AI/Ollama).

2 Opis technologiczny proponowanego rozwiązania

2.1 Architektura ogólna

Aplikacja oparta jest na wzorcu MVC (Model–View–Controller) w środowisku **ASP.NET Core** (.NET 10). Warstwa prezentacji realizowana jest przez widoki Razor (`.cshtml`); logika biznesowa znajduje się w kontrolerach i serwisach; dane utrwalane są w relacyjnej bazie **MySQL 8** przez **Entity Framework Core** (Pomelo provider).



Rysunek 1: Uproszczony schemat architektury systemu *meow*.

2.2 Stos technologiczny

Tabela 1: Technologie wykorzystane w projekcie.

Warstwa	Technologia
Backend	ASP.NET Core MVC, C#
Baza danych	MySQL 8.0, EF Core (Code First)
Cache	Redis 7 (StackExchange.Redis)
API	REST, JSON, Swagger/OpenAPI (Swashbuckle)
Auth (WWW)	Sesje ASP.NET Core, BCrypt
Auth (API)	JWT Bearer
Frontend	Razor Views, Bootstrap 5, CSS/JS
Lokalizacja	IStringLocalizer, pliki .resx (PL/EN)
Mailing	MailKit + Brevo API (prod.) / Mailpit (dev.)
Konteneryzacja	Docker, Docker Compose
Hosting	Railway (https://meow-app-production.up.railway.app)
CI	GitHub Actions (dotnet build)

2.3 Model danych

Główne encje ORM (LibraryDbContext):

- **Book** — tytuł, autor, gatunek, ceny, ilość w sklepie,
- **Egzemplarz** — egzemplarz biblioteczny (numer inwentarzowy, stan),
- **Klient, User** — dane klienta i konto logowania,
- **Wypożyczenie, Platnosc** — wypożyczenia i kary,
- **Zamowienie** — zamówienia sklepowe.

Migracje EF Core stosowane są przy starcie aplikacji (`Database.Migrate()`).

2.4 Warstwa API i cache

Publiczne endpointy REST (prefix `/api`):

- `GET /api/books` — katalog z opcjonalnym filtrem `?q=` i `?gatunek=`,
- `GET /api/books/{id}` — szczegóły książki,
- `POST /api/auth/login` — uwierzytelnienie JWT,
- `POST /api/books/refresh-cache` — unieważnienie cache (dev/admin).

Serwis `BookCatalogCacheService` buforuje wyniki katalogu w Redisie (klucze z prefiksem `meow:`), co ogranicza obciążenie bazy przy częstych odczytach (np. live search w sklepie).

2.5 Uwierzytelnianie i autoryzacja

- **Panel WWW:** sesja po logowaniu (`HttpContext.Session`), atrybut `[MeowAuthorize(Rola = "Admin")]` na akcjach administracyjnych.
- **API mobilne:** token JWT (HS256, issuer/audience w `appsettings.json`), ważność 8 h, claim `klient_id` dla powiązania z profilem.

Hasła przechowywane są jako skróty BCrypt.

2.6 Lokalizacja i interfejs

Interfejs obsługuje języki **polski** (domyślny) i **angielski**. Przełącznik kultury zapisuje wybór w ciasteczku (`CookieRequestCultureProvider`). Teksty UI w plikach `Resources/SharedResources`. Responsywność (RWD) zapewnia `wwwroot/css/responsive.css` oraz komponenty Bootstrap 5.

2.7 Integracje zewnętrzne (stan implementacji)

- **Redis, Swagger, mailing** — zaimplementowane. Cache Redis opcjonalny (fallback: `DistributedMemoryCache`). Maile potwierdzeń zamówień: lokalnie **Mailpit** (port 8025), produkcja **Brevo Transactional API** (`Smtplib.ApiKey`, HTTPS — wymagane na Railway z powodu blokady portu SMTP).
- **Ollama (AI)** — szkielet (`AITestController`, `AIChatController`); wymaga osobnej instalacji Ollama (integracja w zakresie Kyrylo Shchedrin).
- **Qdrant + ONNX** — infrastruktura Docker i `VectorSearchService`; wykorzystanie produkcyjne opcjonalne.

3 Wykaz zadań projektowych z szacowanym czasem

Poniższa tabela przedstawia główne zadania realizacji systemu wraz z orientacyjnym czasem potrzebnym do wykonania (bez przypisania do konkretnych osób).

Zadanie	Czas	Rezultat
Analiza wymagań i projekt architektury	6–10 h	Diagram encji, podział MVC/API
Konfiguracja repozytorium Git, Docker, MySQL	5–8 h	<code>docker-compose.yml</code> , migracje EF
Moduł sklepu (katalog, koszyk, checkout)	16–24 h	<code>ShopController</code> , widoki sklepu
Moduł biblioteki (wypożyczenia, zwroty, rezerwacje)	16–22 h	<code>RentalsController</code> , panel zwrotów
Panel administracyjny (książki, klienci, dashboard)	12–18 h	<code>BooksController</code> , <code>ClientsController</code>
REST API + Swagger/OpenAPI	8–12 h	<code>Controllers/Api/</code> , <code>/swagger</code>

Zadanie	Czas	Rezultat
Cache Redis	3–5 h	BookCatalogCacheService
Uwierzytelnianie (sesje + JWT)	6–10 h	AccountController, AuthApiController
Lokalizacja PL/EN	10–14 h	SharedResources.*.resx
Mailing (MailKit / Brevo API)	3–5 h	EmailService
Interakcje asynchroniczne (live search)	2–4 h	live-search.js
RWD i Bootstrap 5	5–8 h	responsive.css, layout
Integracja AI (Ollama)	8–12 h	AITestController (w trakcie)
CI/CD i dokumentacja wdrożenia	3–6 h	.github/workflows/ci.yml, DEPLOY.md
Hosting produkcyjny (Railway)	3–6 h	https://meow-app-production.up.railway.app
Raporty finansowe admina	3–5 h	ReportsController, Views/Reports/Index.cshtml
Testy, poprawki, sprawozdanie	8–12 h	Stabilna wersja demo
Łącznie (szacunek)	118–182 h	

4 Podział zadań w zespole

Tabela 3: Orientacyjny podział obowiązków (gałąź `filip/aktualna-wersja`).

Osoba	Zakres
Filip Jędrzykowski	MVC (sklep, checkout, wypożyczenia), lokalizacja PL/EN, panel admina, raporty finansowe, mailing (Brevo API), hosting Railway, seed bazy, poprawki UI
Kyryło Shchedrin	Repozytorium GitHub, integracja AI (Ollama, AITestController), wczesna architektura projektu
Nikola Pisarska	Współpraca przy realizacji wymagań PPSI I, testy i dokumentacja

5 Instrukcja uruchomienia systemu

5.1 Wymagania wstępne

- Docker Desktop (Windows/macOS) lub Docker Engine + Compose (Linux),
- Porty wolne: 5000 (WWW), 3307 (MySQL), 6379 (Redis), 8025 (Mailpit), opcjonalnie 6333 (Qdrant),

- Klon repozytorium:

```
git clone https://github.com/04Sqv3r/s4-ppsi1.git
cd s4-ppsi1
git checkout filip/aktualna-wersja    # lub main po merge
```

5.2 Uruchomienie lokalne

W katalogu projektu:

```
docker compose up --build
```

Po zbudowaniu kontenerów:

- Strona WWW: <http://localhost:5000>
- Dokumentacja OpenAPI (Swagger UI): <http://localhost:5000/swagger>
- Specyfikacja JSON: <http://localhost:5000/swagger/v1/swagger.json>

Pierwsze konto: zarejestruj użytkownika przez `/Account/Register`, następnie zaloguj się. Przy starcie aplikacji `DatabaseSeeder` uzupełnia katalog książek (jeśli baza jest pusta). Rolę administratora można nadać w bazie:

```
UPDATE Users SET Rola = 'Admin' WHERE Login = 'twoj_login';
```

(Użytkownik `filip` otrzymuje rolę `Admin` automatycznie, jeśli istnieje w bazie.)

Zatrzymanie:

```
docker compose down
```

5.3 Uruchomienie zdalne (produkcja)

5.3.1 Docker Compose (VPS)

```
docker compose -f docker-compose.yml \
-f docker-compose.prod.yml up -d --build
```

Wymagane zmienne środowiskowe:

- `ConnectionStrings__DefaultConnection`
- `ConnectionStrings__Redis`
- `Jwt__Key` (min. 32 znaki)
- opcjonalnie: `Smtp__ApiKey`, `Smtp__From` (Brevo) — patrz `docs/SMTP.md`

5.3.2 Railway (zalecane dla demo)

Publiczny adres wdrożenia zespołu: <https://meow-app-production.up.railway.app>

1. Utwórz projekt na <https://railway.app> (deploy z GitHub lub CLI).
2. Dodaj usługę **MySQL** (Redis opcjonalny).
3. W aplikacji webowej ustaw m.in.:
 - `ASPNETCORE_URLS=http://0.0.0.0:8080`
 - `ConnectionString__DefaultConnection` — z panelu MySQL
 - `Jwt__Key` — min. 32 znaki
 - `Smtp__ApiKey` — klucz Brevo API (`xkeysib-...`)
 - `Smtp__From` — zweryfikowany nadawca w Brevo
4. Railway wykrywa `Dockerfile` i buduje obraz automatycznie.
5. W zakładce *Networking* wygeneruj publiczny URL.
6. Dokumentacja OpenAPI: <https://meow-app-production.up.railway.app/swagger>

Szczegóły: plik `DEPLOY.md` w repozytorium.

5.4 Pipeline CI

Przy pushu na gałąź `main` uruchamiany jest workflow GitHub Actions (`.github/workflows/ci.yml`): `dotnet restore`, `dotnet build`, `dotnet publish`.

6 Dokumentacja OpenAPI

Specyfikacja API generowana jest automatycznie przez **Swashbuckle.AspNetCore** i udostępniana w formacie OpenAPI 3.0 (tytuł dokumentu: *meow — API biblioteki i sklepu*). W Swagger UI skonfigurowano serwery produkcyjny (Railway) i lokalny (`localhost:5000`) oraz schemat autoryzacji **Bearer** (JWT).

6.1 Dostęp do dokumentacji

Tabela 4: Adresy dokumentacji API.

Zasób	URL
Swagger UI (lokalnie)	http://localhost:5000/swagger
OpenAPI JSON (lokalnie)	http://localhost:5000/swagger/v1/swagger.json
Swagger UI (Railway)	https://meow-app-production.up.railway.app/swagger
OpenAPI JSON (Railway)	https://meow-app-production.up.railway.app/swagger/v1/swagger

Po wdrożeniu produkcyjnym te same ścieżki działają pod publicznym adresem hosta (wymaganie: dokumentacja OpenAPI dołączona do hostowanej aplikacji).

6.2 Endpointy

Tabela 5: Publiczne endpointy REST API (wersja v1).

Metoda	Ścieżka	Opis
GET	/api/books	Lista książek. Parametry: q (frazą), gatunek. Cache Redis.
GET	/api/books/{id}	Szczegóły książki po identyfikatorze.
POST	/api/auth/login	Logowanie; body JSON: { "login", "haslo"}; odpowiedź: token, login, rola, klientId.
POST	/api/books/refresh-cache	Czyści cache katalogu (204 No Content).

W Swagger UI mogą być widoczne także endpointy pomocnicze spoza warstwy /api (np. /AITest/Ask, /Culture/Set/{culture}) — główna dokumentacja REST dla aplikacji mobilnej obejmuje wyłącznie ścieżki z prefiksem /api.

6.3 Przykład wywołania

Logowanie:

```
curl -X POST http://localhost:5000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"login":"user","haslo":"haslo123"}'
```

Katalog z wyszukiwaniem:

```
curl "http://localhost:5000/api/books?q=wiedzmin"
```

Autoryzacja JWT (nagłówek):

```
Authorization: Bearer <token>
```

7 Podsumowanie

System *meow* realizuje zintegrowany model biblioteki i sklepu internetowego z panelem administracyjnym, raportami finansowymi oraz warstwą REST API gotową pod aplikację mobilną. Rozwiązanie konteneryzowane (Docker) umożliwia jednolite uruchomienie lokalne i zdalne; dokumentacja OpenAPI dostępna jest przez Swagger UI pod ścieżką /swagger.

Aplikacja produkcyjna działa pod adresem <https://meow-app-production.up.railway.app>. Mailing potwierdzeń zamówień zrealizowano przez Brevo Transactional API (weryfikacja nadawcy w panelu Brevo).

Element opcjonalny w trakcie rozbudowy: pełna integracja Ollama (moduł AI).